

Tourist Recommender System - Deliverables Summary

Changes Made

✓ Updated Module II

Changed from: Twitter/Big Data Analysis

Changed to: Google Reviews API

Reason: Google Reviews API is more practical and accessible for an undergraduate final year project. It provides:

- Easy integration with Google Places API (already being used)
 - Authentic user reviews and ratings
 - Real popularity metrics
 - Better documentation and support
 - No need for complex big data infrastructure
-

📦 All Deliverables

1. Complete Database ERD

📄 File: [tourist_db_erd.png](#) (286 KB)

What it shows:

- All 21 database tables with complete field listings
- Primary keys (🔑) and foreign keys (🔗) clearly marked
- Relationships between tables with cardinality (1:N, N:N)
- Color-coded by module for easy understanding
- Professional layout suitable for project documentation

Use for:

- Project report (Figure X: Database Schema)
 - Technical documentation
 - Database implementation reference
 - Explaining system design to reviewers
-

2. System Modules Diagram (PNG)

📄 File: [tourist_modules_diagram.png](#) (124 KB)

What it shows:

- High-level system architecture with 5 main modules
- Data flow between modules with labeled arrows
- Integration with Google Places API and Google Reviews API
- Database components linked to each module
- Legend showing which tables belong to each module
- Algorithm notes (K-means, GA, TTDP, OPTW)

Use for:

- Project presentation slides
 - System overview in report introduction
 - Explaining architecture to non-technical audience
 - Quick reference for implementation
-

3. System Modules Diagram (Draw.io)

 File: `tourist_system_modules.drawio` (21 KB)

What it is:

- Editable vector diagram in draw.io XML format
- Can be opened and modified in draw.io (free tool)
- Contains all modules, databases, APIs, and flows
- Fully customizable colors, text, and layout

Use for:

- Making custom versions for different audiences
- Creating presentation-specific diagrams
- Adding your own notes and annotations
- Exporting to different formats (PDF, SVG, PNG)

How to edit:

1. Go to <https://app.diagrams.net/>
 2. Open the file
 3. Edit modules, colors, text as needed
 4. Export to your preferred format
-

4. Database Documentation

 File: `database_documentation.md` (7.3 KB)

What it contains:

- Detailed explanation of all table groups
- Purpose of each table
- Relationships explained in plain English
- Design decisions and rationale
- Database size estimates
- Critical tables for each module
- Implementation notes

Use for:

- Understanding database structure
 - Writing database section in report
 - Reference during implementation
 - Explaining choices to project guide
-

5. Quick Reference Guide

 File: `quick_reference.md` (6.5 KB)

What it contains:

- Table relationships in simple format
- Data flow through the system (step-by-step)
- Common SQL query examples
- Foreign key cascade behavior
- Performance considerations
- Index strategy

Use for:

- Quick lookup during coding
- Understanding system workflow
- Writing SQL queries
- Database optimization

6. Draw.io Usage Guide

 File: `drawio_usage_guide.md` (7.9 KB)

What it contains:

- How to open and edit the .drawio file
- Instructions for customizing the diagram
- Export options (PNG, PDF, SVG, JPG)
- Tips for academic presentations
- Keyboard shortcuts
- Troubleshooting common issues

Use for:

- Learning how to use draw.io
 - Customizing diagrams for your needs
 - Creating multiple diagram versions
 - Preparing presentation materials
-

Recommended Usage for Your Project

For Project Report

Chapter 3: System Design

1. Include `tourist_modules_diagram.png` as "Figure 3.1: System Architecture"
2. Include `tourist_db_erd.png` as "Figure 3.2: Database Schema"
3. Reference `database_documentation.md` when writing database design explanation

Chapter 4: Implementation

- Use `quick_reference.md` to explain data flow
- Reference table relationships when discussing code

For Presentation

Slide 1-2: System Overview

- Use `tourist_modules_diagram.png`
- Highlight the 4 main modules

Slide 3-4: Database Design

- Use `tourist_db_erd.png`

- Zoom into specific table groups as needed
- Or create simplified versions using draw.io file

Slide 5: Algorithms

- Export specific sections from draw.io
- Show K-means clustering flow
- Show GA optimization flow

For Viva/Defense

Question: "Explain your system architecture" → Show modules diagram and walk through the flow

Question: "How do the modules interact?" → Point to arrows in modules diagram, explain data flow

Question: "Explain your database design" → Show ERD and highlight key tables and relationships

Question: "Why did you choose this approach?" → Reference design decisions in documentation



System Flow Summary

Step-by-Step Process

1. User Registration (Module I)

- User creates account and sets preferences
- System analyzes interests
- Tables: `users`, `user_preferences`, `user_interests`

2. POI Collection (Module II)

- Fetch POIs from Google Places API
- Get reviews from Google Reviews API
- Calculate popularity scores
- Tables: `pois`, `poi_categories`, `poi_opening_hours`

3. Trip Planning

- User selects destination and dates
- System calculates travel times between POIs
- Tables: `trips`, `destinations`, `travel_times`

4. Clustering (Module III)

- K-means groups POIs by day
- Each day gets geographically close POIs
- Tables: `clusters`, `cluster_pois`

5. Optimization (Module IV)

- Genetic Algorithm optimizes visit order
- Respects time windows and constraints
- Tables: `itineraries`, `itinerary_pois`, `ga_executions`

6. Presentation

- User sees optimized itinerary
- Can view on map with timings
- Can get restaurant suggestions

7. Feedback (Evaluation)

- User rates POIs and itinerary
- System collects P1-P16 metrics
- Tables: `poi_user_ratings`, `itinerary_feedback`

Technical Stack Recommendations

Based on your database design, here's what you'll need:

Backend

- **Language:** Python 3.8+
- **Web Framework:** Flask or FastAPI
- **Database:** MySQL 8.0+ or MariaDB 10.5+
- **ORM:** SQLAlchemy (optional but recommended)

Algorithms

- **K-means:** scikit-learn library
- **Genetic Algorithm:** DEAP library or custom implementation
- **Distance calculations:** geopy library

APIs

- **Google Places API:** For POI data
- **Google Reviews API:** For popularity analysis
- **Google Distance Matrix API:** For travel times

Frontend

- **Framework:** React or Vue.js
- **Maps:** Google Maps JavaScript API

- **UI Components:** Material-UI or Bootstrap

Tools

- **Database Management:** MySQL Workbench or DBeaver
 - **API Testing:** Postman
 - **Version Control:** Git + GitHub
-



Key Points for Report Writing

Database Design Section

Strengths to highlight:

1. **Normalized Design:** Tables follow 3NF, minimal redundancy
2. **Scalability:** Indexes on key columns for performance
3. **Flexibility:** Supports multiple travel modes, time windows
4. **Completeness:** Captures all research paper requirements
5. **Evaluation Ready:** Built-in feedback mechanisms

Design Decisions to explain:

1. Why separate `cluster_pois` and `itinerary_pois`
2. Why pre-calculate `travel_times` instead of real-time API calls
3. Why use Google Reviews API instead of Twitter
4. Why store GA execution logs
5. Why use views for common queries

Implementation Challenges

Be prepared to discuss:

1. Handling API rate limits and costs
 2. Optimizing GA performance for real-time results
 3. Managing time zone conversions
 4. Dealing with incomplete POI data
 5. User privacy and data security
-



Next Steps

Phase 1: Database Setup (Week 1-2)

1. Set up MySQL database
2. Run the schema SQL file
3. Create indexes
4. Test with sample data

Phase 2: API Integration (Week 3-4)

1. Get Google API keys
2. Implement POI fetching
3. Implement reviews collection
4. Implement distance matrix calculation
5. Set up API caching

Phase 3: Algorithm Implementation (Week 5-7)

1. Implement K-means clustering
2. Implement Genetic Algorithm
3. Test fitness function
4. Optimize parameters
5. Log execution metrics

Phase 4: User Interface (Week 8-10)

1. Build user registration/login
2. Create trip planning interface
3. Display itineraries on map
4. Add feedback forms

Phase 5: Testing & Evaluation (Week 11-12)

1. Conduct user testing
 2. Collect P1-P16 survey responses
 3. Analyze results
 4. Compare with research paper
-

Support

If you need any modifications to the diagrams or documentation:

1. Edit the draw.io file for custom versions
2. Modify the Python scripts to regenerate PNGs
3. Update markdown files as your project evolves

Good luck with your final year project! 🎓